

A COMPREHENSIVE COURSE GUIDE

React Frontend Architecture

The Complete A-to-Z Course Guide

From core concepts to production-grade architecture — components, state, hooks, routing, data layers, performance, testing, and deployment, with a full glossary of terms.

AUDIENCE	Beginners to advanced front-end engineers
FORMAT	Self-paced reference + glossary
COVERAGE	Fundamentals → Architecture → Production

Table of Contents

Part I — Foundations

1. Introduction to React & the Frontend Landscape	4
2. Environment Setup & Tooling	5
3. JSX & Rendering Fundamentals	6
4. Components, Props & Composition	7
5. State & the Component Lifecycle	8

Part II — Core React Mechanics

6. Hooks Deep Dive	9
7. Context API & Prop Drilling Solutions	11
8. Event Handling, Forms & Refs	12
9. Side Effects, Data Fetching & Suspense	13

Part III — Architecture & Scale

10. Frontend Architecture Patterns	14
11. Project & Folder Structure Strategies	16
12. State Management at Scale	17
13. Routing & Navigation Architecture	19
14. Styling Architecture	20
15. TypeScript in React Architecture	21

Part IV — Production Engineering

16. Performance Optimization	22
17. Testing Strategy	24
18. Security Best Practices	25
19. Build Tools, Bundling & Server Rendering	26
20. CI/CD, Deployment & Monitoring	27

Part V — Reference

21. Architecture Checklist & Best Practices Summary	28
22. Glossary of Terms (A–Z)	29

Part I — Foundations

1
.
0

SECTION 1

Introduction to React & the Frontend Landscape

React is a declarative, component-based JavaScript library for building user interfaces, originally released by Facebook (now Meta) in 2013. Rather than manipulating the DOM directly, React lets you describe what the UI should look like for a given state, and it efficiently updates the browser to match that description using a virtual DOM and a reconciliation algorithm.

Why React Dominates Frontend Architecture

- **Declarative UI:** You describe the end state, not the step-by-step DOM mutations.
- **Component model:** UIs are decomposed into independent, reusable, composable building blocks.
- **Unidirectional data flow:** Data flows down through props; events flow up through callbacks, making state changes predictable.
- **Massive ecosystem:** Routing, state management, styling, testing, and meta-frameworks (Next.js, Remix) all build on React's primitives.
- **Cross-platform reach:** React Native and React Native Web extend the same mental model to mobile.

Where React Fits in the Frontend Landscape

Category	Examples	React's Role
UI Libraries	React, Vue, Svelte	Component-based, virtual-DOM diffing
Meta-frameworks	Next.js, Remix, Gatsby	Add routing, SSR/SSG, data loading on top of React
State Tools	Redux, Zustand, Jotai, Recoil	Manage state outside or alongside component tree
Styling	CSS Modules, Tailwind, styled-components	Compose visual design with components
Build Tools	Vite, Webpack, esbuild, Turbopack	Bundle, transpile, and optimize React apps

Key Idea

Architecture is not about picking the 'best' library — it's about choosing patterns that keep an application predictable, testable, and maintainable as it grows from a handful of components to hundreds.

2 . 0 SECTION 2

Environment Setup & Tooling

A solid architecture starts with a clean, fast, and reproducible development environment. Modern React projects are almost universally scaffolded with Vite for client apps, or a meta-framework CLI (Next.js, Remix) when SSR/SSG is needed.

Scaffolding a Project

```
# Vite (fast, recommended for SPAs)
npm create vite@latest my-app -- --template react-ts
cd my-app && npm install && npm run dev

# Next.js (recommended when SSR/SSG/routing-out-of-the-box is needed)
npx create-next-app@latest my-app --typescript
```

Essential Tooling Stack

Tool	Purpose
Node.js (LTS)	JavaScript runtime for the build toolchain
Vite / Webpack	Module bundler and dev server
ESLint	Static code analysis and style enforcement
Prettier	Automated code formatting
TypeScript	Static typing layer over JavaScript
React DevTools	Browser extension for inspecting component trees
Husky + lint-staged	Pre-commit hooks for quality gates

Recommended package.json Scripts

```
"scripts": {
  "dev": "vite",
  "build": "tsc -b && vite build",
  "lint": "eslint . --max-warnings 0",
  "test": "vitest run",
  "preview": "vite preview"
}
```

JSX & Rendering Fundamentals

JSX is a syntax extension that lets you write HTML-like markup inside JavaScript. It compiles to `React.createElement()` calls (or the newer automatic JSX runtime) that produce plain JavaScript objects describing the UI tree.

```
function Greeting({ name }) {  
  return (  
    <div className="card">  
      <h2>Hello, {name}!</h2>  
      {name === 'Admin' && <Badge label="Admin" />}  
    </div>  
  );  
}
```

Rendering Rules to Internalize

- JSX expressions must return a **single root element** (or a Fragment `<>...</>`).
- Curly braces `{ }` embed any JavaScript expression — variables, ternaries, function calls.
- `class` becomes `className`; `for` becomes `htmlFor`.
- Lists require a stable, unique key prop so React can track identity across re-renders.
- React batches state updates within event handlers for performance (and, since React 18, across async boundaries too).

The Virtual DOM & Reconciliation

On every state change, React builds a new virtual DOM tree, diffs it against the previous tree (reconciliation), and applies the minimal set of real DOM mutations needed. This diffing uses heuristics — same component type at the same position is assumed to represent continuity, which is why keys matter so much in lists.

4 SECTION 4 0

Components, Props & Composition

Components are the atomic unit of React architecture. A well-designed component does one thing, accepts data through props, and communicates upward through callbacks.

```
type ButtonProps = {
  label: string;
  onClick: () => void;
  variant?: 'primary' | 'secondary';
};

function Button({ label, onClick, variant = 'primary' }: ButtonProps) {
  return (
    <button className={`btn btn-${variant}`} onClick={onClick}>
      {label}
    </button>
  );
}
```

Composition Over Configuration

Rather than building one component with dozens of boolean props, React favors composition: passing components as children or slots. This keeps components flexible without exploding their prop surface.

```
function Card({ children }: { children: React.ReactNode }) {
  return <div className="card">{children}</div>;
}

// Usage
<Card>
  <CardHeader title="Profile" />
  <CardBody>...</CardBody>
</Card>
```

Component Types

Type	Description
Presentational	Renders UI from props only; no business logic or fetching
Container	Owns state/data fetching; delegates rendering to presentational children
Compound	A set of components that share implicit state (e.g. Tabs.Root, Tabs.Item)
Higher-Order Component (HOC)	A function that wraps a component to inject extra behavior
Render Props	A component that accepts a function-as-child to share logic

5 SECTION 5 0

State & the Component Lifecycle

State is data that changes over time and causes a component to re-render. Understanding the lifecycle — mount, update, unmount — is essential for managing subscriptions, timers, and fetches correctly.

```
function Counter() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    document.title = `Count: ${count}`;
  }, [count]);

  return <button onClick={() => setCount(c => c + 1)}>{count}</button>;
}
```

Lifecycle Phases (Function Components)

Phase	Hook Equivalent	Typical Use
Mount	<code>useEffect(fn, [])</code>	Subscriptions, initial fetch, analytics
Update	<code>useEffect(fn, [deps])</code>	React to prop/state changes
Unmount	Cleanup function returned from <code>useEffect</code>	Clear timers, cancel requests, unsubscribe
Common Pitfall Forgetting an effect's cleanup function is the single most common source of memory leaks and 'setState on unmounted component' warnings in React apps.		

Part II — Core React Mechanics

6
.
0

SECTION 6

Hooks Deep Dive

Hooks let function components use state, context, and lifecycle features without writing classes. They must be called unconditionally, in the same order, on every render.

Hook	Purpose
useState	Local component state
useEffect	Side effects synced to dependencies
useLayoutEffect	Synchronous effect before browser paint (measuring DOM)
useContext	Read a Context value without prop drilling
useReducer	Complex state transitions via reducer pattern
useMemo	Memoize an expensive computed value
useCallback	Memoize a function reference to avoid unnecessary re-renders
useRef	Persist a mutable value across renders without re-rendering
useId	Generate stable unique IDs for accessibility attributes
useTransition	Mark state updates as non-urgent for smoother UI
useDeferredValue	Defer re-rendering of a value until the UI is idle
useSyncExternalStore	Subscribe to external stores (used by Redux, Zustand internally)

Custom Hooks: Extracting Reusable Logic

A custom hook is just a function whose name starts with use and that may call other hooks. This is the primary mechanism for sharing stateful logic across components without HOCs or render props.

```
function useDebounceValue<T>(value: T, delay = 300): T {
  const [debounced, setDebounce] = useState(value);

  useEffect(() => {
    const id = setTimeout(() => setDebounce(value), delay);
    return () => clearTimeout(id);
  }, [value, delay]);

  return debounced;
}
```

Rules of Hooks

- Only call hooks at the top level — never inside loops, conditions, or nested functions.
- Only call hooks from React function components or other custom hooks.

- The dependency array of `useEffect/useMemo/useCallback` must list every reactive value used inside.

7 SECTION 7 0

Context API & Prop Drilling Solutions

Prop drilling — passing props through several intermediate components that don't need them — is a common architectural smell. Context provides a way to share values (theme, auth, locale) across a subtree without manual threading.

```
const ThemeContext = createContext<'light' | 'dark'>('light');

function App() {
  return (
    <ThemeContext.Provider value="dark">
      <Dashboard />
    </ThemeContext.Provider>
  );
}

function ThemedPanel() {
  const theme = useContext(ThemeContext);
  return <div className={`panel-${theme}`}>...</div>;
}
```

Architecture Note

Context is not a replacement for full state management. It re-renders every consumer whenever its value changes, so it's best for low-frequency, broadly shared values (theme, auth, locale, feature flags) — not high-frequency state like form input or live data streams.

Event Handling, Forms & Refs

React wraps native DOM events in a SyntheticEvent for cross-browser consistency. Forms can be controlled (React owns the input value) or uncontrolled (the DOM owns it, accessed via refs).

```
function LoginForm() {
  const [email, setEmail] = useState('');
  const passwordRef = useRef<HTMLInputElement>(null);

  function handleSubmit(e: React.FormEvent) {
    e.preventDefault();
    console.log(email, passwordRef.current?.value);
  }

  return (
    <form onSubmit={handleSubmit}>
      <input value={email} onChange={e => setEmail(e.target.value)} />
      <input ref={passwordRef} type="password" />
    </form>
  );
}
```

For complex forms, libraries like React Hook Form or Formik reduce re-renders and centralize validation logic (often paired with a schema validator like Zod or Yup).

9 . 0 SECTION 9

Side Effects, Data Fetching & Suspense

Data fetching is one of the most architecturally significant decisions in a React app. Raw `useEffect` fetching is error-prone (race conditions, no caching, no retries) — production apps typically use a dedicated data layer.

Approach	Best For
<code>useEffect</code> + <code>fetch</code>	Tiny apps, prototypes, learning fundamentals
TanStack Query (React Query)	Caching, retries, background refetch, pagination
SWR	Lightweight stale-while-revalidate caching
RTK Query	Apps already standardized on Redux Toolkit
Server Components / loaders	Next.js / Remix apps fetching on the server
Suspense for data	Coordinating loading states across nested components

```
function useUser(id: string) {
  return useQuery({
    queryKey: ['user', id],
    queryFn: () => fetch(`/api/users/${id}`).then(r => r.json()),
    staleTime: 60_000,
  });
}
```

Suspense lets a component 'wait' on async data or code, showing a fallback until it resolves — pairing naturally with React Server Components and modern data-fetching libraries.

Part III — Architecture & Scale

1
0
.
0
SECTION 10

Frontend Architecture Patterns

As applications grow, ad-hoc component trees become hard to navigate. Several established architectural patterns help keep large React codebases coherent.

1. Layered (N-Tier) Architecture

- **UI layer** — presentational components, purely visual.
- **Application/state layer** — hooks, stores, business logic.
- **Domain layer** — types, validation, core entities.
- **Data/infrastructure layer** — API clients, caching, persistence.

2. Feature-Sliced Design (FSD)

Organizes code by business feature rather than technical type, with strict import rules between layers (app → pages → widgets → features → entities → shared). This scales well for large teams working on independent product areas.

3. Atomic Design

Level	Description	Example
Atoms	Smallest indivisible UI elements	Button, Input, Label
Molecules	Small groups of atoms working together	SearchBar (Input + Button)
Organisms	Complex, distinct sections of UI	Header, ProductCard
Templates	Page-level layout skeletons	DashboardTemplate
Pages	Templates filled with real data	DashboardPage

4. Micro-Frontends

Splits a large application into independently deployable frontend units (often owned by different teams), composed at runtime via Module Federation, iframes, or web components. Powerful for very large orgs, but adds operational complexity — best reserved for genuinely large, multi-team products.

Choosing a Pattern

Small app / single team: simple feature-folder structure is enough. Medium app: layered or feature-sliced design. Large, multi-team product: feature-sliced design or micro-frontends, with a shared design system underneath.

1

1

SECTION 11

.

0

Project & Folder Structure Strategies

Two dominant strategies exist: organizing by technical type, or by feature/domain. Most scaled React codebases converge on feature-based organization with a shared kernel of reusable building blocks.

```
src/
  app/           # App shell, providers, routing config
  pages/         # Route-level components
  features/
    auth/
      components/
      hooks/
      api.ts
      types.ts
    cart/
      components/
      hooks/
      store.ts
  shared/
    ui/          # Design-system primitives (Button, Input...)
    lib/         # Generic utilities
    hooks/       # Cross-feature hooks
  entities/      # Core domain types (User, Product...)
```

- Keep feature folders self-contained: components, hooks, API calls, and types that belong to one feature live together.
- Avoid deep relative imports (`../../../../shared/ui`) — configure path aliases (`@/shared/ui`).
- Co-locate tests next to the code they test, not in a separate mirrored tree.
- Enforce import boundaries with ESLint rules so features don't reach into each other's internals.

1

2

.

0

SECTION 12

State Management at Scale

Not all state is equal. A core architectural skill is correctly classifying state and routing it to the right tool — using a heavyweight global store for everything is itself an anti-pattern.

State Type	Description	Typical Tool
Local UI state	Toggle, input value, hover	useState / useReducer
Shared UI state	Theme, modal visibility across a subtree	Context
Server/cache state	Data fetched from an API	TanStack Query, SWR, RTK Query
Global client state	Cart, auth session, app-wide flags	Zustand, Redux Toolkit, Jotai
URL state	Filters, pagination, tab selection	Search params / router state
Form state	Input values, validation, touched fields	React Hook Form, Formik

Comparing Global State Libraries

Library	Model	Strengths
Redux Toolkit	Single store, reducers, actions	Predictable, strong devtools, time-travel debugging
Zustand	Minimal hook-based store	Tiny API, low boilerplate, good performance
Jotai	Atomic state primitives	Fine-grained reactivity, composable atoms
Recoil	Atoms + selectors graph	Derived state, good for complex dependency graphs
MobX	Observable objects	Automatic reactivity, less boilerplate, OOP-friendly

Architectural Principle

Push state as close as possible to where it's used. Lift it only when two or more components genuinely need to share it. Reach for a global store only when state must be accessed from many unrelated parts of the tree.

1

3

SECTION 13

.

0

Routing & Navigation Architecture

Routing determines how URLs map to UI. React Router remains the most widely used client-side router for SPAs; Next.js and Remix provide file-system-based routing with built-in data loading.

```
const router = createBrowserRouter([
  {
    path: '/',
    element: <RootLayout />,
    children: [
      { index: true, element: <Home /> },
      { path: 'products/:id', element: <ProductPage />, loader: productLoader },
      { path: 'dashboard', element: <ProtectedRoute><Dashboard /></ProtectedRoute> },
    ],
  },
]);
```

Architectural Considerations

- **Code splitting:** lazy-load route components with React . lazy to keep initial bundles small.
- **Route guards:** centralize authentication/authorization checks in layout or wrapper components, not scattered per page.
- **Nested layouts:** share persistent UI (nav, sidebar) across child routes without remounting.
- **Data loaders:** fetch route data before render (React Router loaders, Next.js server components) to avoid loading waterfalls.

1
4
.
0

SECTION 14

Styling Architecture

Approach	Description	Trade-offs
CSS Modules	Locally scoped CSS files per component	Simple, zero runtime; less dynamic theming
Tailwind CSS	Utility-first classes composed in markup	Fast iteration; verbose className strings
styled-components / Emotion	CSS-in-JS with component-scoped styles	Dynamic styling; runtime cost, larger bundles
Vanilla Extract / Panda CSS	Zero-runtime CSS-in-JS, compiled at build time	Type-safe, no runtime cost; newer ecosystem
Design system (e.g. Radix + Tailwind)	Pre-built accessible primitives + utility styling	Consistency and accessibility out of the box

Most production teams settle on one primary approach and layer a design system (tokens for color, spacing, typography) on top, so visual decisions are made once and reused everywhere.

1

5

SECTION 15

.

0

TypeScript in React Architecture

TypeScript adds static typing to props, state, context, and API responses, catching an entire class of bugs before runtime and making large codebases self-documenting.

```
interface User {  
  id: string;  
  name: string;  
  role: 'admin' | 'member';  
}  
  
function UserBadge({ user }: { user: User }) {  
  return <span className={`badge badge-${user.role}`}>{user.name}</span>;  
}
```

- Type API responses explicitly; never trust any at the network boundary — validate with Zod or io-ts for runtime safety too.
- Use discriminated unions to model UI states cleanly (loading | error | success).
- Prefer type for unions/utility composition and interface for extensible object shapes — pick one convention per team.
- Generate types from your backend schema (OpenAPI/GraphQL codegen) to keep frontend and backend in sync automatically.

Part IV — Production Engineering

1
6
.
0
SECTION 16

Performance Optimization

Performance work in React generally falls into two buckets: reducing unnecessary re-renders, and reducing the amount of code/data shipped to the browser.

Reducing Re-renders

Technique	When to Use
React.memo	Pure presentational component re-rendering with same props
useMemo	Expensive derived calculations
useCallback	Stable function references passed to memoized children
Virtualization (react-window)	Long lists/tables — render only visible rows
Splitting context	Avoid one giant context causing broad re-renders
State colocation	Move state down to the smallest subtree that needs it

Reducing Bundle & Load Time

- **Code splitting:** React . lazy + dynamic imports per route or heavy feature.
- **Tree-shaking:** use ES modules and avoid importing entire libraries for one function.
- **Image optimization:** modern formats (WebP/AVIF), responsive sizes, lazy loading.
- **Server rendering / static generation:** ship pre-rendered HTML for faster first paint.
- **Prefetching:** preload likely-next routes/data on hover or viewport entry.

Measuring Performance

- **React DevTools Profiler:** identify which components re-render and why.
- **Lighthouse / Web Vitals:** LCP, INP (replacing FID), and CLS as core UX metrics.
- **Bundle analyzers:** visualize what's actually shipped to users.

1

7

SECTION 17

.

0

Testing Strategy

A healthy React testing strategy follows the testing pyramid: many fast unit tests, a moderate number of integration tests, and a few high-value end-to-end tests.

Layer	Tool	Tests
Unit	Vitest / Jest	Pure functions, hooks in isolation, reducers
Component / Integration	React Testing Library	Rendered components, user interactions, accessibility
End-to-End	Playwright / Cypress	Full user flows across real or mocked backends
Visual Regression	Chromatic / Percy	Catch unintended UI changes

```
test('shows error when login fails', async () => {
  render(<LoginForm onSubmit={mockFailingSubmit} />);
  await userEvent.type(screen.getByLabelText('Email'), 'test@x.com');
  await userEvent.click(screen.getByRole('button', { name: /sign in/i }));
  expect(await screen.findByText(/invalid credentials/i)).toBeVisible();
});
```

React Testing Library's philosophy — test components the way a user interacts with them (by role, label, and text, not implementation details) — leads to tests that survive refactors.

1 8 . 0

SECTION 18

Security Best Practices

- **XSS prevention:** never use `dangerouslySetInnerHTML` with unsanitized input; React escapes text content by default.
- **Dependency hygiene:** audit packages regularly (`npm audit`), pin versions, avoid abandoned libraries.
- **Secrets:** never embed API keys or secrets in client bundles — all client code is publicly readable.
- **Auth tokens:** prefer `httpOnly` cookies over `localStorage` for session tokens to reduce XSS exposure.
- **CSP headers:** configure Content-Security-Policy at the server/CDN level to restrict script sources.
- **Input validation:** validate on both client (UX) and server (security) — never trust the client alone.

1
9
.
0

SECTION 19

Build Tools, Bundling & Server Rendering

How a React app is built and rendered fundamentally shapes its architecture — particularly the choice between client-side rendering, server-side rendering, static generation, and the newer React Server Components model.

Rendering Strategy	Description	Good For
CSR (Client-Side Rendering)	Empty HTML shell, JS renders everything in-browser	Internal tools, dashboards behind auth
SSR (Server-Side Rendering)	HTML generated per-request on the server	SEO-sensitive, personalized public pages
SSG (Static Site Generation)	HTML pre-built at build time	Marketing pages, blogs, docs
ISR (Incremental Static Regen.)	Static pages regenerated on a schedule/demand	Catalogs with infrequent updates
RSC (React Server Components)	Components render on the server, ship zero JS to client	Data-heavy pages needing minimal client JS

Build Tool Landscape

Tool	Notes
Vite	Native ES modules in dev for near-instant HMR; Rollup-based production build
Webpack	Mature, highly configurable; still common in larger legacy codebases
esbuild / SWC	Extremely fast Go/Rust-based transpilers used under the hood by newer tools
Turbopack	Rust-based bundler used by Next.js for faster local dev

CI/CD, Deployment & Monitoring

Architecture extends beyond code into how it ships and how its health is observed in production.

CI/CD Pipeline Stages

- **Lint & type-check:** fail fast on style and type errors.
- **Test:** run unit, integration, and (on merge) end-to-end suites.
- **Build:** produce optimized production bundles.
- **Preview deploy:** spin up an ephemeral environment per pull request for review.
- **Deploy:** promote to staging, then production, often with canary or blue-green rollout.

Observability

Tool Category	Examples	Purpose
Error tracking	Sentry, Bugsnag	Capture and triage runtime exceptions
Real User Monitoring	Web Vitals, Datadog RUM	Measure real-world performance
Analytics	Amplitude, PostHog, GA4	Understand user behavior and funnels
Feature flags	LaunchDarkly, Unleash	Progressive rollout and kill-switches

Part V — Reference

2
1
.
0

SECTION 21

Architecture Checklist & Best Practices Summary

Use this checklist when reviewing or designing a React application's architecture.

- ■ Components are small, single-purpose, and composable.
- ■ State is classified correctly (local / shared / server / global / URL / form) and routed to the right tool.
- ■ Folder structure is feature-based with clear, enforced import boundaries.
- ■ Data fetching uses a dedicated caching layer (TanStack Query, SWR, RTK Query) rather than raw effects.
- ■ Routes are code-split; protected routes centralize auth checks.
- ■ A consistent styling approach and design system are in place.
- ■ TypeScript types extend to the network boundary with runtime validation.
- ■ Performance is measured (Profiler, Web Vitals), not just assumed.
- ■ Tests follow the pyramid: many unit/component tests, few E2E tests.
- ■ Security basics are covered: no secrets in client code, sanitized HTML, secure token storage.
- ■ CI/CD pipeline enforces lint, type-check, and tests before every deploy.
- ■ Errors and performance are monitored in production, not just in development.

Final Takeaway

Good React architecture is rarely about exotic patterns — it's about consistently applying simple principles (single responsibility, correct state placement, clear boundaries) at every scale, from a single component to a hundred-engineer codebase.

Glossary of Terms (A–Z)

A reference glossary of terms used throughout this course, in alphabetical order.

Atomic Design

A methodology for structuring UI components into atoms, molecules, organisms, templates, and pages, from smallest to largest.

Bundler

A build tool (Vite, Webpack, esbuild) that combines and optimizes JavaScript, CSS, and assets into deployable files.

Code Splitting

Dividing a bundle into smaller chunks that load on demand, reducing the initial JavaScript payload.

Component

A self-contained, reusable unit of UI that accepts props and returns rendered output.

Composition

Building complex UI by combining simpler components, typically via children or slots, rather than configuration props.

Context API

A built-in React feature for sharing values across a component tree without manually passing props at every level.

Controlled Component

A form element whose value is driven entirely by React state, with changes handled via onChange.

CSR (Client-Side Rendering)

Rendering that happens entirely in the browser after an initial, mostly empty HTML shell loads.

CSS-in-JS

An approach where component styles are written and scoped within JavaScript/TypeScript files (e.g. styled-components).

Custom Hook

A reusable function, prefixed with 'use', that encapsulates stateful logic built from other hooks.

Declarative UI

A programming style where you describe the desired end state of the UI rather than the steps to reach it.

Memoization

Caching the result of a computation or a component render to avoid redundant work.

Meta-framework

A framework built on top of a UI library (e.g. Next.js on React) adding routing, SSR, and data conventions.

Micro-Frontend

An architecture splitting a frontend application into independently built and deployed pieces.

Prop Drilling

Passing props through several layers of components that don't use them, just to reach a deeply nested child.

Props

Read-only inputs passed from a parent component to a child component.

Reconciliation

React's algorithm for diffing a new virtual DOM tree against the previous one to compute minimal DOM updates.

Render Props

A pattern where a component accepts a function as a prop (often as children) to share logic and rendering control.

RSC (React Server Components)

Components that render exclusively on the server and send no JavaScript to the client by default.

Route Guard

Logic that restricts access to a route based on conditions such as authentication or permissions.

Selector

A function that derives or extracts a specific slice of state from a larger store.

Side Effect

An operation (data fetching, subscriptions, DOM manipulation) that interacts with the world outside of rendering.

Dependency Array

The second argument to `useEffect/useMemo/useCallback` listing values that trigger re-execution when changed.

Discriminated Union

A TypeScript pattern using a common literal field to distinguish between several related object shapes.

Feature-Sliced Design (FSD)

An architecture methodology organizing code by business feature with strict layer-based import rules.

Fragment

A React wrapper (`<>...`) that groups multiple elements without adding an extra DOM node.

Hook

A function (e.g. `useState`, `useEffect`) that lets function components use React features like state and lifecycle.

Higher-Order Component (HOC)

A function that takes a component and returns a new component with added behavior or props.

Hydration

The process of attaching React's event handlers and state to server-rendered HTML in the browser.

Immutability

The practice of never mutating state directly, instead creating new objects/arrays to trigger re-renders.

ISR (Incremental Static Regeneration)

A Next.js rendering strategy that regenerates static pages in the background after a set interval.

JSX

A syntax extension allowing HTML-like markup to be written inside JavaScript, compiled to function calls.

Key Prop

A special prop used in lists to help React identify which items changed, were added, or were removed.

Lazy Loading

Deferring the loading of a component, route, or asset until it is actually needed.

SSG (Static Site Generation)

Pre-rendering HTML pages at build time rather than on each request.

SSR (Server-Side Rendering)

Generating HTML on the server for each request, then hydrating it on the client.

State

Data that changes over time within a component and triggers re-rendering when updated.

Store

A centralized object holding application state, typically used by global state libraries like `Redux` or `Zustand`.

Suspense

A React feature that lets components 'wait' for asynchronous content and show a fallback UI while waiting.

SyntheticEvent

React's cross-browser wrapper around native DOM events, normalizing behavior across browsers.

Tree-Shaking

A build optimization that removes unused code from the final bundle.

Uncontrolled Component

A form element whose value is managed by the DOM itself, typically accessed via a `ref`.

Unidirectional Data Flow

An architectural principle where data flows one way (parent to child) and events flow back up via callbacks.

Virtual DOM

An in-memory representation of the UI that React uses to compute efficient updates to the real DOM.

Virtualization

Rendering only the visible portion of a long list or table to improve performance.

Web Vitals

A set of metrics (LCP, INP, CLS) used by Google to measure real-world user experience quality.

Lifting State Up

Moving state to the closest common ancestor of components that need to share it.

XSS (Cross-Site Scripting)

A security vulnerability where malicious scripts are injected into and executed within a trusted web page.

End of Course Guide — React Frontend Architecture: The Complete A-to-Z Guide.